DESENVOLVIMENTO MOBILE · MÓDULO 19 · TRILHA KOTLIN · NÍVEL AVANÇADO

Desenvolvimento Multiplataforma

Flutter, React Native e Kotlin Multiplatform: arquitetura, código comparado e critérios de escolha

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar como Flutter, React Native e Kotlin Multiplatform chegam ao Android e ao iOS.
- Ler e escrever uma tela simples em Dart/Flutter e em TypeScript/React Native.
- Estruturar um projeto Kotlin Multiplatform com código comum e específico.
- Mapear os conceitos de Compose para os equivalentes em cada framework.
- Escolher a abordagem adequada a um projeto e a uma equipe.

Você já sabe mais do que imagina

Os três frameworks usam **UI declarativa**, **estado** e **componentes**, os mesmos conceitos dos Módulos 05–08.

Roteiro

Neste módulo

- Fundamentos do desenvolvimento multiplataforma
- Panorama na prática
- Flutter
- React Native
- Kotlin Multiplatform
- Como escolher
- Fechamento

Fundamentos do desenvolvimento multiplataforma

O problema: muitas plataformas, uma equipe

Por que existe o multiplataforma

Android e iOS têm linguagens, ferramentas e componentes diferentes. Dois apps nativos significam duas bases de código, duas equipes e o dobro de testes para cada funcionalidade.

Desenvolvimento multiplataforma

Escrever uma vez parte (ou todo) o código e executá-lo em mais de um sistema, aceitando algum compromisso em desempenho, aparência ou acesso ao hardware.

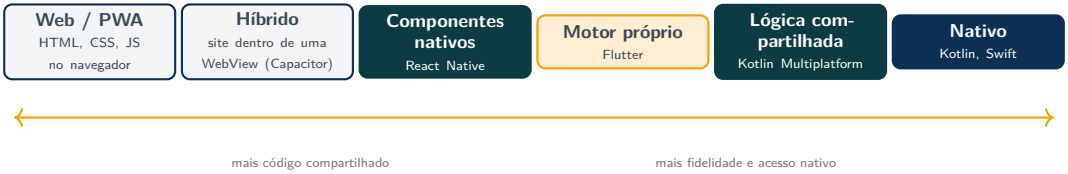
Um pouco de história

Java prometia "escreva uma vez, rode em qualquer lugar" nos anos 1990. No mobile, vieram os apps híbridos com WebView (PhoneGap/Cordova, 2009), o React Native (2015), o Flutter (2017–2018) e o Kotlin Multiplatform (estável em 2023).

Atenção

Nenhuma abordagem elimina o conhecimento de cada plataforma: permissões, publicação, ciclo de vida e integrações nativas continuam diferentes.

O espectro das abordagens



	O que se compartilha	Quem desenha a tela	Compromisso principal
Web/PWA e híbrido	tudo	o navegador (HTML/CSS)	desempenho e acesso limitado ao hardware
React Native	UI e lógica em JS/TS	componentes nativos de cada sistema	ponte entre JS e o nativo
Flutter	UI e lógica em Dart	o motor do Flutter, pixel a pixel	tamanho do app; aparência própria
KMP	lógica (e opcionalmente a UI)	nativa ou Compose Multiplatform	UI pode exigir código por plataforma

Como o código executa em cada abordagem

Tecnologia	Execução no aparelho
PWA / híbrido	JavaScript interpretado e compilado sob demanda (JIT) pelo motor do navegador
React Native	JavaScript pré-compilado em *bytecode* e executado pelo motor Hermes
Flutter	Dart compilado antes (AOT) para código de máquina ARM
KMP (Android)	Kotlin → *bytecode* → DEX, executado pelo ART
KMP (iOS)	Kotlin/Native compilado via LLVM para binário nativo
Nativo	Kotlin no ART; Swift compilado via LLVM

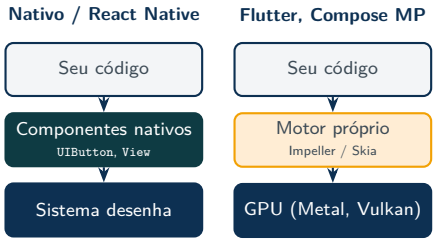
JIT e AOT

JIT (*just-in-time*): compila durante a execução; flexível, permite recarregar código. **AOT** (*ahead-of-time*): compila antes da distribuição; inicia mais rápido e tem desempenho previsível.

Por que o iOS importa aqui

A App Store não permite que apps gerem e executem código de máquina novo em tempo de execução (exceto o motor do navegador). Por isso o Flutter usa AOT no iOS e o React Native usa um interpretador de *bytecode*.

Quem desenha os pixels?



Componentes nativos

A tela usa os controles do sistema: aparência e comportamento idênticos aos de outros apps e atualizados junto com o sistema.

Motor próprio

O framework desenha cada pixel numa tela em branco: aparência idêntica em todas as plataformas e controle total de animações, mas precisa reimplementar o comportamento de cada controle.

Analogia

Montar a casa com móveis de loja (nativos) ou fabricar os próprios móveis (motor próprio).

Conversando com o nativo: pontes, canais e FFI

Mecanismo	Como funciona
Troca de mensagens	chamadas serializadas e assíncronas entre os dois lados (canais de plataforma do Flutter, ponte antiga do React Native)
Interface direta	chamada de funções nativas sem serializar, inclusive síncrona (FFI do Dart, JSI do React Native)
Geração de código	a interface é declarada uma vez e o código dos dois lados é gerado (Pigeon, Turbo Modules, exportação para Swift do KMP)

O custo de atravessar

Cada chamada por mensagem converte os dados para bytes, troca de *thread* e converte de volta. Para chamadas raras (abrir a câmera) não importa; para milhares por segundo (áudio, imagens), use interfaces diretas.

FFI — *Foreign Function Interface*

Mecanismo para chamar funções escritas em outra linguagem, em geral bibliotecas C, compartilhando memória.

Progressive Web Apps

PWA

Site que se comporta como app: pode ser instalado na tela inicial, funciona offline e recebe notificações. Base: um **manifesto** (nome, ícones, cores) e um ***service worker*** (script que intercepta requisições e gerencia o cache).

Vantagens

Uma única base web, sem loja e sem revisão, atualização instantânea, link compartilhável.

Limites

- Acesso a hardware depende do navegador: Web Bluetooth e Web NFC não existem no Safari.
- No iOS, notificações exigem que o usuário instale o site na tela inicial.
- Menos presença nas lojas e menos integração com o sistema.

Dica

Para um painel de consulta do LabMonitor no computador do laboratório, uma versão web basta; para BLE e NFC, o app nativo ou multiplataforma é necessário.

Decidindo com uma matriz ponderada

Critério	Peso	Nativo	Flutter	RN	KMP
Atender Android e iOS	3	2	5	5	4
Acesso a BLE, NFC, câmera	3	5	4	4	5
Equipe já sabe a linguagem	2	4	3	2	4
Tempo até a 1ª versão	2	2	5	4	3
Manutenção em 5 anos	1	4	4	3	4
Total ponderado		37	**47**	42	45

Notas de 1 a 5 ilustrativas: app novo, equipe que conhece Kotlin. Com um app Kotlin já existente, o critério "reaproveitar código" favoreceria o KMP.

Matriz de decisão ponderada

Lista os critérios, dá um peso a cada um conforme sua importância, atribui notas às opções e soma nota × peso.

Atenção

O resultado depende dos pesos: discuta-os com a equipe e o cliente **antes** de dar as notas, para não ajustar a matriz ao resultado desejado.

Além do celular: telas e formatos

Responsivo e adaptativo

Responsivo: o mesmo layout se estica ou reorganiza conforme o espaço. **Adaptativo**: layouts diferentes para classes de tela e para cada forma de entrada (toque, mouse, teclado, controle remoto).

Classe	Largura	Exemplos
Compacta	< 600 dp	celular em pé
Média	600–839 dp	tablet em pé, dobrável aberto
Expandida	≥ 840 dp	tablet deitado, desktop

Classes de janela do Material Design.

Outros formatos

Relógios, TVs, carros (Android Auto, CarPlay), painéis embarcados e quiosques. Cada um muda a distância de leitura, a forma de entrada e o tempo de atenção do usuário.

Dica

Projete pelo **espaço disponível**, não pelo tipo de aparelho: uma janela de desktop pode ser estreita como um celular.

Panorama na prática

Três estratégias de compartilhamento



Flutter

Compartilha **tudo**; a interface é desenhada pelo próprio motor, idêntica nas plataformas.

React Native

Compartilha **tudo**; os componentes viram widgets nativos de cada sistema.

KMP

Compartilha **o que fizer sentido**: só a lógica ou também a UI.

Comparação

	Flutter	React Native	Kotlin Multiplatform
Linguagem	Dart	JavaScript / TypeScript	Kotlin
Mantenedor	Google	Meta e comunidade	JetBrains (com apoio do Google)
Interface	widgets próprios desenhados pelo motor	componentes nativos	Compose Multiplatform ou nativa
Desenvolvimento	*hot reload*	*fast refresh*	*hot reload* (Compose)
Acesso nativo	*plugins* e *platform channels*	módulos nativos (Turbo Modules)	direto (`expect`/`actual`)
Outras plataformas	web, desktop	web (parcial)	desktop, web, servidor
Para quem sabe Kotlin	aprender Dart	aprender JS/TS e React	continua em Kotlin

Dica

Todos são usados em produção por grandes empresas. Escolha pelo **contexto** (equipe, código existente, requisitos), não por modismo.

Flutter

Flutter: tudo é widget

lib/painel.dart

```dart
class Painel extends StatefulWidget {
  const Painel({super.key});
  @override
  State<Painel> createState() => _PainelState();
}

class _PainelState extends State<Painel> {
  double temp = 24.0;                  // estado

  @override
  Widget build(BuildContext context) => Center(
    child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('${temp.toStringAsFixed(1)} °C'),
        ElevatedButton(
          onPressed: () => setState(() => temp += 0.5),
          child: const Text('Simular'),
        ),
      ]),
  );
}
```

Conceitos

- `StatelessWidget`: sem estado próprio.
- `StatefulWidget` + `State`: com estado; `setState` pede um novo `build`.
- Layout com widgets: `Column`, `Row`, `Padding`, `Center`.

Compose ↔ Flutter

`@Composable`	`Widget.build`
`mutableStateOf`	`setState`
`Modifier.padding`	`Padding(...)`
`LazyColumn`	`ListView.builder`
`ViewModel`	`ChangeNotifier`/Bloc

Flutter: ecossistema e estrutura

Começando

```
flutter create labmonitor_flutter
cd labmonitor_flutter
flutter run          # r = hot reload
flutter pub add http riverpod
flutter build appbundle
```

pubspec.yaml (trecho)

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  flutter_riverpod: ^2.5.0
```

Dica

Pacotes em `https://pub.dev`. Confira as versões atuais antes de copiar.

Necessidade	Pacotes comuns
Estado	Provider, Riverpod, Bloc
Rede	`http`, `dio`
Banco local	`drift`, `sqflite`
Firebase	FlutterFire
BLE	`flutter_blue_plus`
MQTT	`mqtt_client`
Navegação	`go_router`

Pontos fortes e limites

UI idêntica em todas as plataformas, *hot reload* rápido e ótima documentação. O motor embutido deixa o APK maior que o nativo.

React Native

React Native: componentes e hooks

app/index.tsx (Expo)

```tsx
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function Painel() {
  const [temp, setTemp] = useState(24.0);   // estado

  return (
    <View style={styles.centro}>
      <Text style={styles.valor}>{temp.toFixed(1)} °C</Text>
      <Button title="Simular"
        onPress={() => setTemp(t => t + 0.5)} />
    </View>
  );
}

const styles = StyleSheet.create({
  centro: { flex: 1, alignItems: 'center',
            justifyContent: 'center' },
  valor: { fontSize: 48, fontWeight: 'bold' },
});
```

Conceitos

- Componentes são funções que devolvem JSX.
- `useState`, `useEffect`: *hooks* (como `remember` e `LaunchedEffect`).
- Estilo parecido com CSS e layout *flexbox*.
- Nova arquitetura (Fabric, Turbo Modules, JSI).

Expo

Caminho recomendado para começar: `npx create-expo-app`, rotas por arquivos e o app *Expo Go*.

Kotlin Multiplatform

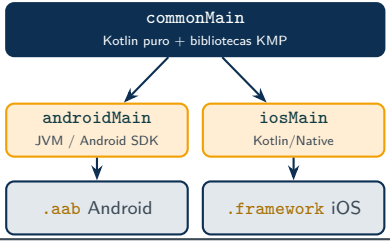
Estrutura de um projeto KMP

Projeto KMP

```
labmonitor-kmp/
|-- composeApp/
|   `-- src/
|       |-- commonMain/    UI Compose comum
|       |-- androidMain/   MainActivity
|       `-- iosMain/       MainViewController
|-- shared/
|   `-- src/
|       |-- commonMain/    modelos, repositórios,
|       |                  casos de uso, Ktor
|       |-- androidMain/   implementações Android
|       `-- iosMain/       implementações iOS
`-- iosApp/                projeto Xcode (Swift)
```

Dica

O **Kotlin Multiplatform Wizard** (`https://kmp.jetbrains.com`) gera essa estrutura pronta.



Camada	Bibliotecas KMP
Rede	Ktor Client + kotlinx.serialization
Banco	Room (KMP), SQLDelight, DataStore
Estado	ViewModel (lifecycle KMP), coroutines
DI	Koin
UI	Compose Multiplatform

expect e actual

commonMain/Plataforma.kt

```
expect fun nomePlataforma(): String
```

androidMain/Plataforma.android.kt

```
actual fun nomePlataforma() =
  "Android ${Build.VERSION.SDK_INT}"
```

iosMain/Plataforma.ios.kt

```
import platform.UIKit.UIDevice

actual fun nomePlataforma() =
  UIDevice.currentDevice.systemName() + " " +
  UIDevice.currentDevice.systemVersion
```

commonMain: uso

```
Text("Rodando em ${nomePlataforma()}")
```

O mecanismo

`expect` declara **o quê** no código comum; cada plataforma fornece o `actual`, **como**, usando suas APIs nativas (Android SDK ou frameworks da Apple).

Prefira interfaces

Para dependências maiores (BLE, sensores), declare uma `interface` no código comum e injete implementações por plataforma com Koin. Fica mais fácil de testar.

Atenção

Compilar e rodar a parte iOS exige **macOS com Xcode**.

Compartilhando a lógica do LabMonitor

shared/commonMain/ClimaRepository.kt

```kotlin
class ClimaRepository(private val http: HttpClient) {
  suspend fun atual(lat: Double, lon: Double): ClimaDto =
    http.get("https://api.open-meteo.com/v1/forecast") {
      parameter("latitude", lat)
      parameter("longitude", lon)
      parameter("current", "temperature_2m")
    }.body()
}

class ClimaViewModel(
  private val repo: ClimaRepository
) : ViewModel() {                  // lifecycle KMP
  val estado = MutableStateFlow<Double?>(null)
  fun carregar() = viewModelScope.launch {
    estado.value = repo.atual(-2.53, -44.30)
      .current.temperatura
  }
}
```

O que mudou em relação ao Android

Quase nada: Retrofit → **Ktor**, Hilt → **Koin**. DTOs, `Flow`, `ViewModel`, casos de uso e testes continuam iguais.

Migração gradual

Um app Android existente pode mover **primeiro** os modelos e casos de uso para um módulo `shared`, sem reescrever a UI. É a forma mais segura de adotar KMP.

Como escolher

A mesma tela nos três mundos

Conceito	Compose (Kotlin)	Flutter (Dart)	React Native (TS)
Unidade de UI	função `@Composable`	classe `Widget`	função componente
Estado local	`remember { mutableStateOf() }`	`State` + `setState`	`useState`
Efeito ao entrar	`LaunchedEffect`	`initState`	`useEffect`
Lista longa	`LazyColumn`	`ListView.builder`	`FlatList`
Layout	`Column`/`Row`/`Box`	`Column`/`Row`/`Stack`	`View` + flexbox
Estado da tela	`ViewModel` + `StateFlow`	Riverpod / Bloc	Context, Zustand, Redux
Assíncrono	coroutines / `Flow`	`Future` / `Stream`	`Promise` / `async`

Os conceitos se transferem: quem domina um framework declarativo aprende o outro em semanas.

Critérios de decisão

Perguntas a fazer

1. Precisa de iOS? Quando?
2. Que linguagem a equipe já domina?
3. Há código Android/Kotlin existente?
4. O app depende muito de hardware (BLE, câmera, sensores)?
5. A UI deve ser idêntica ou seguir o padrão de cada sistema?
6. Há também web ou desktop no horizonte?

Cenário	Sugestão
Só Android, hardware intenso	Kotlin nativo
App Android existente indo para iOS	KMP
Equipe web/React	React Native
UI de marca idêntica em tudo, equipe nova	Flutter
Protótipo rápido para as duas lojas	Flutter ou RN + Expo

Para o LabMonitor

Como o app usa BLE e já está em Kotlin, o caminho natural para o iOS é **KMP**: compartilhar modelos, repositórios e casos de uso, e escolher entre Compose Multiplatform e SwiftUI para a UI.

Fechamento

Resumo

Resumo

- Flutter desenha a própria UI com Dart; React Native usa componentes nativos com JS/TS; KMP compartilha Kotlin e deixa a UI a critério.
- Os três são declarativos: estado, componentes e efeitos se mapeiam diretamente para Compose.
- KMP usa `commonMain` + `expect`/`actual` e bibliotecas como Ktor, Room, Koin e Compose Multiplatform.
- iOS sempre exige macOS e Xcode para compilar e publicar.
- A escolha depende da equipe, do código existente, do hardware e das plataformas-alvo.

Glossário (1/2)

Termo	Significado
Multiplataforma	código escrito uma vez e executado em mais de um sistema
JIT / AOT	compilação durante a execução / antes da distribuição
Ponte e JSI	camadas que ligam o JavaScript ao código nativo no React Native
Renderização própria	o framework desenha os pixels (Flutter, Compose Multiplatform)
Componentes nativos	a interface usa os widgets do sistema (React Native, SwiftUI)
KMP	Kotlin Multiplatform: compartilha código Kotlin entre plataformas
`expect/actual`	declaração no código comum e implementação em cada plataforma
Compose Multiplatform	Compose para Android, iOS, desktop e web
Kotlin/Native	compila Kotlin para código de máquina, sem JVM (iOS)

Glossário (2/2)

Termo	Significado
`commonMain`	conjunto de código compartilhado de um projeto KMP
XCFramework	pacote binário que o Xcode consome com o código KMP
Turbo Modules	módulos nativos da nova arquitetura do React Native
Expo	plataforma e ferramentas que simplificam projetos React Native
PWA	site instalável, com manifesto e *service worker*
FFI	chamada direta a funções C, na mesma memória
Matriz ponderada	técnica de decisão que soma notas multiplicadas por pesos

Questões de revisão

1. Compare Flutter, React Native e KMP quanto ao que é compartilhado e a quem desenha a interface.
2. Por que apps iOS não podem usar JIT, e como o Kotlin/Native resolve isso?
3. Para que servem `expect` e `actual`? Dê um exemplo no LabMonitor.
4. Que partes do LabMonitor você colocaria em `commonMain`, e quais ficariam em cada plataforma?
5. Qual a diferença entre falar com o nativo por ponte (ou canal) e por FFI?
6. Quando uma PWA basta, e quando é preciso um app nativo ou multiplataforma?
7. Monte uma matriz ponderada para levar o LabMonitor ao iOS. Que critérios e pesos você usaria?
8. Por que os conceitos de Compose se transferem com facilidade para Flutter e React Native?

Exercícios

Exercício 1 — Flutter

Reescreva em Flutter a tela de temperatura da Open-Meteo (Módulo 11) com `http` e um `FutureBuilder` mostrando carregando, sucesso e erro.

Exercício 2 — React Native

Crie com Expo uma lista de equipamentos com `FlatList` e uma tela de detalhe usando o roteamento por arquivos.

Exercício 3 — Comparação

Meça, para o app do exercício 1 e para o equivalente em Compose, o tamanho do APK release e o tempo de abertura. Discuta os resultados.

LabMonitor — Etapa 19

Extraia modelos, DTOs e o `VerificarAlertasUseCase` para um módulo `shared` KMP com testes em `commonTest`. Opcional: tela de painel em Compose Multiplatform rodando no desktop.

Para aprofundar

- Flutter: `https://docs.flutter.dev` Flutter para devs Android: `https://docs.flutter.dev/get-started/flutter-for/android-devs`
- React Native: `https://reactnative.dev` Expo: `https://docs.expo.dev`
- Kotlin Multiplatform: `https://kotlinlang.org/docs/multiplatform.html`
- KMP no Android: `https://developer.android.com/kotlin/multiplatform`
- Compose Multiplatform: `https://www.jetbrains.com/compose-multiplatform/`
- Ktor Client: `https://ktor.io/docs/client-create-new-application.html`

Respostas das questões de revisão

1. Flutter compartilha interface e lógica e desenha os pixels com motor próprio. React Native compartilha interface e lógica em JS/TS e usa componentes nativos. KMP compartilha a lógica e, se quiser, a interface (Compose Multiplatform); a UI pode continuar nativa.
2. A App Store não permite gerar e executar código de máquina novo em tempo de execução. O Kotlin/Native compila antes (AOT) e entrega um *framework* binário ao Xcode.
3. `expect` declara no código comum algo que cada plataforma implementa com `actual`. Exemplo: `expect class FonteBle`, implementada sobre a API Bluetooth do Android e sobre o CoreBluetooth do iOS.
4. Comum: modelos, regras de alerta e reserva, repositórios, cliente HTTP (Ktor), banco (Room ou SQLDelight) e *view models*. Por plataforma: BLE, permissões, notificações e as telas, se a UI for nativa.
5. Ponte ou canal trocam mensagens serializadas, em geral assíncronas, entre dois mundos; o FFI chama funções nativas diretamente, na mesma memória, inclusive de forma síncrona e com menos custo.
6. PWA basta para consulta, formulários e painéis sem hardware especial nem presença em loja. App é necessário com BLE, NFC, trabalho em segundo plano ou alto desempenho.
7. Por exemplo: reaproveitar o código Kotlin (peso 3), acesso a BLE (3), aparência nativa no iOS (2), curva de aprendizado (2) e manutenção (2). Com o app em Kotlin e dependente de BLE, o KMP tende a vencer.
8. Os três são declarativos: UI como função do estado, composição, estado elevado e fluxo unidirecional. Mudam a sintaxe e as APIs, não o modelo mental.

Fim do Módulo 19

Próximo: Módulo 20 — Tópicos avançados e projeto final: IA no dispositivo, segurança, monetização e carreira.